Chapter 5

The µML Object Model

Contents

5. The µML Object Model

The *µML Object Model* is a directed acyclic graph, which may be created by hand during the design phase, or may be created automatically by model transformation. When sketched by hand, an Object Model is drawn as an object graph ordered by existence dependency. When created by model transformation, an Object Model is obtained from a *µML Class Model* by first normalising that model, and then translating all normalised relationships into simple references.

The purpose of the Object Model is to represent the set of normalised data entities in a given system, and the normalised relationships between them. Each object consists of simply-typed attributes and object-typed references, which refer to another object in the graph. References may be qualified further to denote *kind-of, part-of* or *made-of* relationships. Together, the attributes and references are known as the object's properties; and any subset of these properties may uniquely identify the object.

5.1 Elements of the Object Model

The Object Model consists of the following concepts:

- **Basic Type** – is an uninterpreted entity, denoting a primitive type or simple datatype that has a name, such as: *Boolean, Integer, Character, Real, String, Date,* or *Money*.
- **Object Type** – is a structured entity, denoting an object, a piece of modelled data or information, that has a name and a set of properties, which are attributes and references.

The Object Model also consists of the following features:

- **Property** – a named and typed feature owned by an object, which possibly serves as one of the object's identifiers.
- **Attribute** – a kind of property denoting a named attribute, whose type is a *Basic Type*.
- **Reference** – a kind of property denoting a named reference, whose type is an *Object Type*. A reference may be qualified to denote a *kind-of, part-of* or *made-of* relationship.

The Object Model also has the following regions:

- **Diagram** – is an implicit kind of region enclosing everything in the Object Model. The diagram is the root container for an Object Model.

5.2 Notation for the Object Model

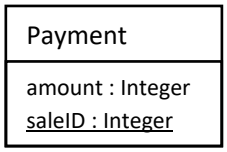
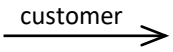
Object Type: denotes a structured information entity. Drawn as a rectangle containing the name of the object, with a drop-down partition listing its attributes. Any identifying attributes are underlined.	
Reference: denotes an existence dependency of the source object on the target object. Drawn as a transition, labelled with the name of the target. Any identifying references are underlined.	

Figure 5.1: Notation for the Object Model

Figure 5.1 illustrates the notation used for concepts and connectives in the µML Object Model. It adopts standard µML conventions in the concepts and features used. The icon for an *Object Type* is the expanded version of the simple rectangle used to display any object in overview. It has one drop-down partition listing the object's attributes. The arrow for a *Reference* is the same as that for a transition. It is

labelled with the name of the target, which is either the camel-case name of the type, or some other distinguishing role-name (necessary if two distinct references connect the same source and target). Both attributes and references may be identifying – eventually, each *Object Type* will have one or more identifying properties.

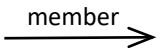
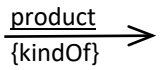
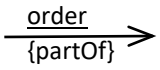
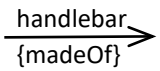
Plain Reference: denotes an existence dependency of the source upon the target object, which is named on the label. Dependent objects cannot exist without their masters.	
KindOf Reference: denotes a subtype-supertype relationship between the source and target object. The qualification {kindOf} is written after or below the name.	
PartOf Reference: denotes a part-whole relationship between the source and target, in which the part is a material constituent of the whole. Qualified with {partOf}	
MadeOf Reference: denotes a whole-part relationship between the source and target, in which the whole is an assembly of its parts. Qualified with {madeOf}.	

Figure 5.2: Meanings of different kinds of Reference

Figure 5.2 illustrates the notation used for plain and qualified kinds of reference in the µML Object Model. References are features owned by the object in question, rather than connectives linking two objects, hence the visual emphasis of dependency of the source upon the target. The difference in meaning is explained in the rules of the Object Model.

5.3 Rules of the Object Model

The Object Model has the following syntactic and semantic rules:

- **Structured type** – an object type must be a structured information entity, defined in terms of its attributes and references, whose structure is explained in the model (a definition rule).
- **Uninterpreted type** – a basic type is either a primitive type such as *Integer*, or an uninterpreted data type such as *Date*, whose structure is not exposed in the model (a definition rule).
- **Attribute type** – the type of an attribute must be a basic type in the same Object Model (a completeness rule).
- **Reference type** – the type of a reference must be an object type in the same Object Model (a completeness rule).
- **Implicit multiplicity** – every reference is implicitly a many-to-one (or optional-to-one) normalised relationship between the source and target (a semantic rule).
- **Existence dependency** – the default existence dependency of a source upon a target means that the source's lifetime is shorter than the target's lifetime. The target cannot be deleted until the source has been deleted (a semantic rule).
- **Kind-of dependency** – the kind-of dependency of a source upon a target unites the lifetimes of the source and target. The source is a weak subtype entity, identified wholly by its reference to the target. Deleting the target also deletes the source (a semantic rule).
- **Part-of dependency** – the part-of dependency of a source upon a target unites the lifetimes of the source and target. The source is a weak component entity, identified partially by its reference to the target. Deleting the target also deletes the source (a semantic rule).
- **Made-of dependency** – the made-of dependency of a source upon a target means that the lifetimes of the source assembly and target components are independent. If a target component is deleted, the reference in the source is removed (a semantic rule).

- **Identifying properties** – every object type must eventually have at least one identifying property, whether attribute or reference, and may have many. Weak entities (subtype or component) must have an identifying reference to their target, and this is the *only* identifier allowed for a subtype, whereas a component may also have a weak identifying attribute (a definition rule).
- **Implicit boundary** – the diagram is the implicit boundary of the Object Model. Every contained object is part of the system (a completeness rule).

These rules mean that an Object Model can be implemented directly in any object-oriented programming language, since all relationships are resolved to references. The semantics of qualified references must also be preserved by forwards model transformation to a data model. For example, both kinds of weak entity must result in foreign keys with cascading deletion.

5.4 Object Model Examples

Some examples are given below of Object Models. The first example describes a Lending Library. The second example describes a Training Agency.

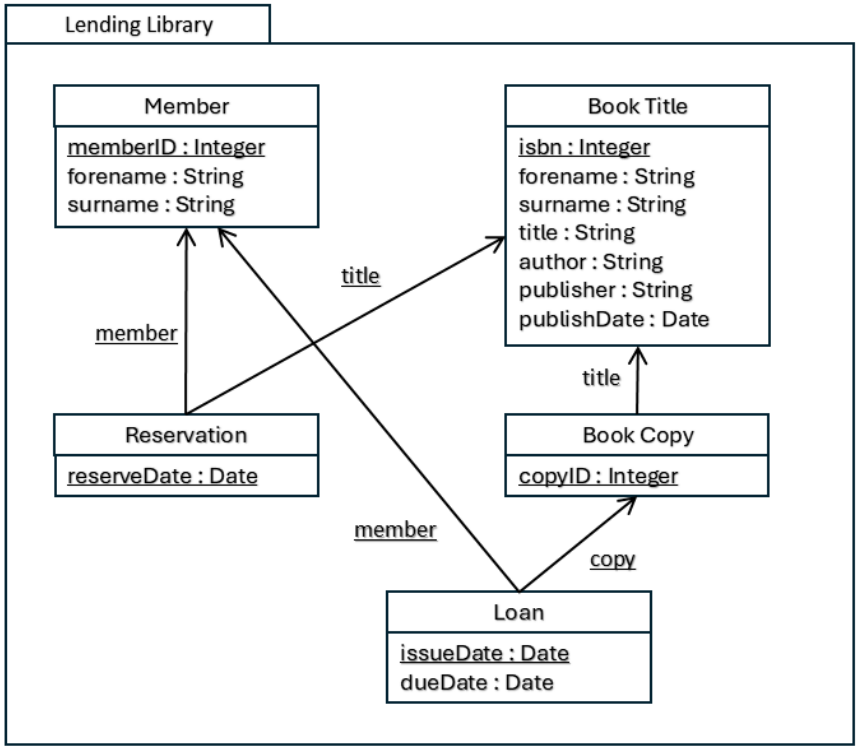


Figure 5.3: Object Model for a Lending Library

Figure 5.3 illustrates simple Object Model for a Lending Library, which issues loans of books to its members. The model contains three basic types {*Integer, String, Date*} given as the types of attributes, and five object types, shown with their attributes and references:

- **Member** – denotes a member of the lending library, identified by a surrogate *memberID* attribute, since neither the forename or surname may serve as identifiers
- **Book Title** – denotes a book in its role as a title, identified by a natural *isbn* attribute. Natural identifiers from the domain are preferred, where these exist.
- **Book Copy** – denotes a book in its role as a copy of a book title, identified by a surrogate *copyID* attribute. We assume this globally identifies a copy.
- **Loan** – denotes the loan of a book (copy) to a member, identified by a combination of its references to a *member* and a *copy*, and also by the *issueDate* attribute.

- **Reservation** – denotes the reservation of a book (title) to a member, identified by a combination of its references to a *member* and a *title*, and also by the *reserveDate* attribute.

Many references are identifying. This is normal for entities that describe a time-bounded relationship between other entities. The reason why Loan has an extra identifying *issueDate* attribute is to distinguish Loans with the same *copy* and *member* on different occasions. This is necessary if Loans are archived; but not strictly necessary if Loan objects are deleted, when the loan is discharged.

Note that the object types in this model may not necessarily be wholly consistent with the objects identified earlier in the Impact Model. A consistency maintenance transformation is needed to update the Impact Model with new objects revealed in the Object Model.

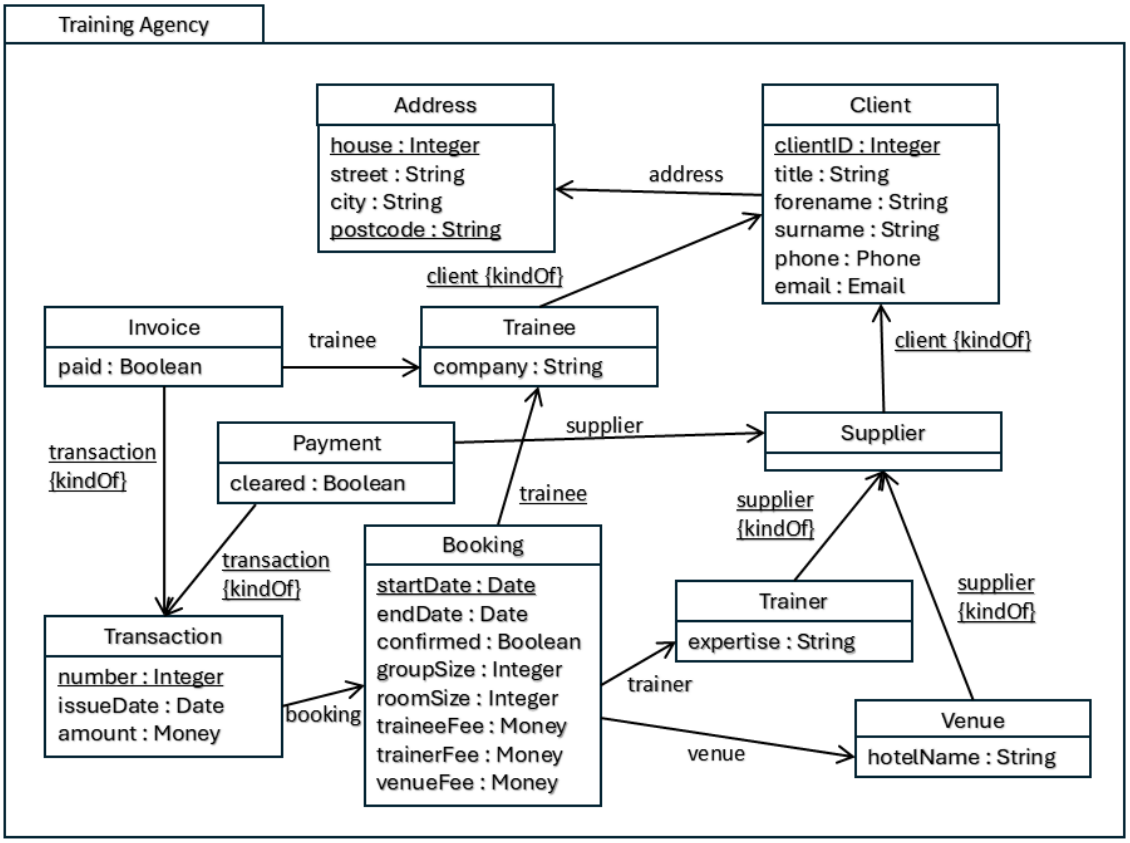


Figure 5.4: Object Model for a Training Agency

Figure 5.4 illustrates a more complex Object Model for a Training Agency, which arranges training sessions for trainees with trainers, booked into different venues. The model contains the seven basic types {*Boolean, Integer, String, Date, Money, Phone, Email*} and ten structured object types, with their attributes and references:

- **Address** – denotes the address of a client and is a root of the existence dependency graph, identified by a pair of natural attributes {*house, postcode*}.
- **Client** – denotes a client in the most general sense, identified by the surrogate *clientID* attribute. Note that this has some attributes with uninterpreted types: Email and Phone.
- **Trainee** – is a kind of Client, identified by its *client {kindOf}* reference.
- **Supplier** – is a kind of Client, identified by its *client {kindOf}* reference.
- **Trainer** – is a kind of Supplier, identified by its *supplier {kindOf}* reference.
- **Venue** – is a kind of Supplier, identified by its *supplier {kindOf}* reference.

- **Booking** – denotes a booking involving a trainee organisation, a trainer and a venue. It is identified by the *trainee* reference and the *startDate* attribute, on the assumption that these two properties are sufficient (no duplicate sessions for the same organisation on the same day).
- **Transaction** – denotes a general money transaction, made in respect of a booking, identified by a natural transaction *number* attribute.
- **Invoice** – is a kind of Transaction, identified by its *transaction {kindOf}* reference. An Invoice is charged to a Trainee.
- **Payment** – is a kind of Transaction, identified by its *transaction {kindOf}* reference. A Payment is made to a polymorphic Supplier of services – either a Trainer, or a Venue.

This Object Model includes some qualified *kindOf* references. These describe the subtype-supertype relationships between different kinds of client, and must therefore all be identifying references. If this model were mapped to a database design, information about a real-world client could be spread over two, or three logical tables.

In the existence dependency graph, Address is the least dependent entity, and Payment and Invoice have the longest dependency chains on other objects. Note again that the object types in this model are not wholly consistent with the objects identified earlier in the Impact Model. A consistency maintenance transformation is needed to update the Impact Model with new objects revealed in the Object Model.

5.5 Object Metamodel

The elements of the µML Object Model may be defined in a metamodel. In figure 5.5, we use a metamodel written in the *ReMoDeL* metamodel definition language [1]. This extends and redefines concepts defined in the ReMoDeL metamodel of the µML Core Model, given in Chapter 2.

```
# The Object Model from muML (micro Modelling Language).  This consists of
# BasicTypes and ObjectTypes.  ObjectTypes encapsulate Properties, which are
# either Attributes or References.  Attributes have BasicTypes, whereas
# References have ObjectTypes and indicate dependency.  Certain References
# are marked as "kindOf", "partOf" or "madeOf" dependencies.  The Object
# Model is an existence dependency graph, in which all relationships have
# been resolved to object dependencies.

metamodel ObjectModel inherit CoreModel {

# Type is a concept denoting any kind of type.  It can be
# cast to a BasicType or ObjectType.  All Types are owned by
# a Diagram.

   concept Type inherit Concept {
      reference owner : Diagram
      operation asBasic : BasicType { self }
      operation asObject : ObjectType { self }
   }

# BasicType denotes a primitive type, or any atomic type
# with no further analysed structure.

   concept BasicType inherit Type {
   }

# ObjectType denotes a structured type, defined with a
# list of properties.  These are either Attributes with
# simple types, or References with object types.  Some
# References may indicate a kindOf dependency, and some
# may indicate a partOf dependency.  The operations that
# return allProperties, allReferences, allAttributes
# exclude any kindOf Reference.

   concept ObjectType inherit Type {
```

```
      component properties : Property[]
      operation attributes : Attribute[] {
         properties.select(prop | prop.isAttribute)
      }
      operation references : Reference[] {
         properties.select(prop | prop.isReference)
      }
      operation identifiers : Property[] {
         properties.select(prop | prop.id)
      }
      operation isAbstract : Boolean {
         owner.objectTypes.exists(obj |
            obj.supertypes.has(self))
      }
      operation isMutable : Boolean {
         properties.exists(prop | prop.isMutable)
      }
      operation supertypes : ObjectType[] {
         self.references.select(ref | ref.kindOf)
            .collect(ref | ref.type)
      }
      operation allAttributes : Attribute[] {
         self.allProperties.select(prop | prop.isAttribute)
      }
      operation allReferences : Reference[] {
         self.allProperties.select(prop | prop.isReference)
      }
      operation allProperties : Property[] {
         self.supertypes.collate(sup | sup.allProperties)
            .append(self.properties.reject(prop |
               prop.isReference and prop.asReference.kindOf))
      }
   }

# TypedFeature is a kind of named Feature with a Type.
# It may be a mutable, or immutable (constant) feature.

   concept TypedFeature inherit Feature {
      reference type : Type
      operation isMutable : Boolean { false }
   }

# Variable is a kind of TypedFeature with any kind of Type.
# A Variable is typically mutable.

   concept Variable inherit TypedFeature {
      operation isMutable : Boolean { true }
   }

# Property is a kind of TypedFeature owned by an ObjectType.
# The Property may be identifying.  A Property reports whether
# it is an Attribute, a Reference or a precursor Reference.

   concept Property inherit TypedFeature {
      reference owner : ObjectType
      attribute id : Boolean
      operation isAttribute : Boolean { false }
      operation isReference : Boolean { false }
      operation asAttribute : Attribute { self }
      operation asReference : Reference { self }
   }

# Attribute is a kind of Property that has a BasicType.  An
# Attribute is mutable so long as it is not identifying.

   concept Attribute inherit Property {
      reference type : BasicType
      operation isAttribute : Boolean { true }
      operation isMutable : Boolean { not id }
      operation isSurrogate : Boolean {
```

```
         id and name.endsWith("ID")
      }
   }

# Reference is a kind of Property that has an ObjectType.
# A Reference is typically immutable after construction.
# A Reference may be marked as kindOf (subtype dependency)
# or partOf (material dependency) or madeOf (assembly).

   concept Reference inherit Property {
      reference type : ObjectType
      attribute kindOf : Boolean
      attribute partOf : Boolean
      attribute madeOf : Boolean
      operation isReference : Boolean { true }
   }

# Diagram is a collection of BasicTypes and ObjectTypes

   concept Diagram inherit Concept {
      component basicTypes : BasicType[]
      component objectTypes : ObjectType[]
   }
}
```

Figure 5.5: Metamodel of the Object Model in the ReMoDeL language.

This provides the necessary elements of the Object Model via a number of intermediate abstractions in the metamodel hierarchy:

- **Type** – is a kind of core Concept owned by a Diagram and generalising over BasicType and ObjectType, and can be viewed as either a BasicType or an ObjectType;
- **BasicType** – is a kind of Type, denoting an uninterpreted basic type;
- **ObjectType** – is a kind of Type, denoting a structured object type having a list of Properties, which can be filtered to show Attributes, References or identifying Properties;
- **TypedFeature** – is a kind of core Feature, having a name and a Type;
- **Variable** – is a kind of TypedFeature that is not used further here;
- **Property** – is a kind of TypedFeature owned by an ObjectType, which is optionally identifying, and which can be viewed as either an Attribute or a Reference;
- **Attribute** – is a kind of Property denoting a simple attribute, having a BasicType type, and is mutable only if it is not identifying;
- **Reference** – is a kind of Property denoting an object reference, having an ObjectType type, and may optionally be qualified as a *kindOf, partOf* or *madeOf* reference;
- **Diagram** – is a kind of Region that contains all the BasicTypes and ObjectTypes of the model.

Note that some of these concepts define many operations, which select or compute different parts of the model. In particular, ObjectType offers operations to select its local attributes and references, but also operations to select all of its attributes and references, including those inherited from supertypes. This flattens the view of the object's properties.

Another of the computed properties is the *isMutable* operation, which determines whether an ObjectType is capable of being updated. This is true, if any of the object's Attributes are also mutable. An Attribute is only mutable if it is not identifying. References are immutable, that is, you cannot update object structure in-place, but you can delete and create a new object structure.

5.6 Model Transformations

The µML Object Model is a source or target in a number of model transformations. Some of these are to ensure completeness of the Object Model, or enforce model consistency across different views, and some are to develop more concrete views of the software system.

- **ObjectToImpactModel** – is a mapping transformation that takes an existing µML Object Model as the source, and from this constructs a target µML Impact Model, with atomic tasks that *create, read, update* and *delete* each object type. This is useful where no prior Impact Model exists, but tasks have synthesized names. The result is strongly complete and observable (see chapter 4).
- **MergeObjectWithImpact** – is a merging transformation that takes an incomplete µML Impact Model and a µML Object Model as the sources, and from this constructs an updated target µML Impact Model, which is both observable and strongly complete, with respect to *create, delete, update* and *inspect* tasks. This retains existing task names, where possible.
- **ToFlatObjectModel** – is a mapping transformation that takes a µML Object Model as the source, and from this constructs a flattened µML Object Model, in which *kindOf* references have been eliminated by pulling down all inherited properties from supertype to subtype. This works best if no polymorphic references are used anywhere in the model, otherwise such references are replicated for each concrete ObjectType.
- **ObjectToDataModel** – is a mapping transformation that takes an existing µML Object Model and from this constructs a target µML Data Model, consisting of database tables, columns, primary keys and foreign keys. Object references are transformed into sets of foreign key columns using a recursive attribute renaming algorithm. The target Data Model translates the dependency structure of the source Object Model exactly (whether this is flat, or not).
- **ImpactObjectToProcess** – is a merging transformation that takes a µML Impact Model and a µML Object Model as the sources, and from this constructs a µML Process Model, in which the processes are decomposed into different kinds of actions that input or output values, fetch or store objects, and create or delete objects.

These transformations are too large to display in full here.

5.7 Differences from UML

There is no direct precursor for the µML Object Model, which is not found in this form in UML. The closest work that influenced our design of the µML Object Model is the Belgian software development method *MERODE* [2, 3, 4] (see chapter 4 for a full explanation of this method). This highlighted the importance of existence dependency when constructing object graphs. Along with the *MERODE* authors, we developed a systematic rule-based procedure for constructing Object Models that were already in 3rd Normal Form (3NF) [5].

In *Merode*, an Existence Dependency Graph (EDG) is constructed, which is populated initially with objects that have no dependency. As further co-ordinating objects are discovered, these are added to the EDG, linking the co-ordinating object as a dependent of their co-ordinated master objects. The design process continues until all found objects are linked in the EDG.

Whereas UML does have the notion of an Object Diagram [6], this is not a type-level diagram, but a snapshot of a collection of instances at a given point in time. The µML Object Model describes design relationships between object types, and is perhaps more similar to a restricted use of the UML Class Diagram, in which the only allowed relationship is a directed association, indicating an owned structural dependency. This does not account for the qualified references in µML, which can only be expressed in the full form of a UML Class Diagram as generalisation, aggregation or composition relationships. In µML, such semantic relationships exist in the µML Class Model, but they are resolved by model transformation in favour of more concrete references in the µML Object Model.

5.8 References

[1] AJH Simons, ReMoDeL Explained (rev 2.2): an introduction to ReMoDeL by example. *Technical Report, 25 January*, University of Sheffield (2023). https://staffwww.dcs.shef.ac.uk/people/A.Simons/remodel/current/ReMoDeL_Explained_25Jan25.pdf

[2] M Snoeck. *Enterprise Information Systems Engineering – The MERODE Approach*, Springer (2014).

[3] M Snoeck, C Michiels and G Dedene. Consistency by construction: The case of MERODE. In MA Jeusfeld, O Pastor Eds., Conceptual modeling for novel application domains, *Lecture Notes in Computer Science*, **2814**, Springer (2003), 105-117.

[4] M Snoeck and G Dedene. Existence Dependency: the key to semantic integrity between structural and behavioral aspects of object types, *IEEE Transactions on Software Engineering*, **24** (24) (1998), 233–251.

[5] AJH Simons, M Snoeck and KSY Hung. Using design patterns to reveal the competence of object-oriented methods in system-level design, *Computer Systems Science and Engineering*, **14** (6), CRL Publishing Ltd. (1999), 343-352.

[6] G Booch, J Rumbaugh, I Jacobson. *The Unified Modeling Language User Guide*, Addison-Wesley (1999).